

car

February 1, 2026

First Step: Import Libraries

- import numpy, pandas, matplotlib, KNeighborsClassifier from sklearn.neighbors and some metrics

```
[85]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, \
    classification_report
```

Second Step: Read csv file

- read the csv file data using pd (pandas)

```
[86]: df = pd.read_csv("car.csv")
```

Third Step: Preview Data

- preview the first 5 rows of data to see the type of the data like categorical and numerical
- also to see the type of output column if discrete or continuous

```
[87]: df.head()
```

```
[87]:   Price Maintenance Cost Doors Persons Luggage Boot Security Acceptability
0  52000             vhigh    2      2      small    low      unacc
1  70400             vhigh    2      2      small    med      unacc
2  88800             vhigh    2      2      small    high     unacc
3  52000             vhigh    2      2        med    low      unacc
4  70400             vhigh    2      2        med    med      unacc
```

Fourth Step: Initialize X

- X is only the input data without the output column

```
[88]: X = df.drop(columns=["Acceptability"], axis=1)
X
```

```
[88]:
```

	Price	Maintenance	Cost	Doors	Persons	Luggage	Boot	Security
0	52000		vhigh	2	2		small	low
1	70400		vhigh	2	2		small	med
2	88800		vhigh	2	2		small	high
3	52000		vhigh	2	2		med	low
4	70400		vhigh	2	2		med	med
...	...		...	...	...		...	
1723	7600		low	5more	more		med	med
1724	9900		low	5more	more		med	high
1725	5300		low	5more	more		big	low
1726	7600		low	5more	more		big	med
1727	9900		low	5more	more		big	high

[1728 rows x 6 columns]

Fifth Step: Initialize Y

- y is only the output column

```
[89]: y = df["Acceptablity"]
      y
```

```
[89]:
```

0	unacc
1	unacc
2	unacc
3	unacc
4	unacc
...	
1723	good
1724	vgood
1725	unacc
1726	good
1727	vgood

Name: Acceptablity, Length: 1728, dtype: object

Sixth Step: Import and Initialize transformers

- based on the data:
 - if we have categorical data import OneHotEncoder to transform them to numerical
 - if we have numerical data import StandardScaler to scale values (so no feature is dominant over the others)

```
[90]: num_features = X.select_dtypes(exclude="object").columns
      cat_features = X.select_dtypes(include="object").columns

      from sklearn.preprocessing import OneHotEncoder, StandardScaler
      from sklearn.compose import ColumnTransformer
```

```
numerical_transformer = StandardScaler()
hot_transformer = OneHotEncoder()

preprocessor = ColumnTransformer ([
    ("OneHotEncoder", hot_transformer, cat_features),
    ("StandardScaler", numerical_transformer, num_features)
])
```

Seventh Step: fit transform X

- the input must be fit transformed using the transformers to apply transformation
- here we will apply the note that to scale after splitting: fit transform for X_train and transform for X_test

```
[91]: # X = preprocessor.fit_transform(X)
```

Eighth Step: split data into training and testing

- we must split data into training and testing
- as a some standards way:
 - training 80%, testing 20%
 - training 70%, testing 30%

```
[92]: from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=0.2,
↪train_size=0.8, random_state=42)
```

Ninth Step: Better than step 7 (either step 7 either this)

- but you can skip it now for the hackathon and do 7
- it is better to transform training and testing like this below instead of fit transform all X
 - fit transform all X is not a good idea, because the model will see the whole X training and testing
 - fit transform all X can lead to data leakage because:
 - * fit_transform calculates the parameters (like mean and variance) from the data and applies the transformation in one step, whereas transform uses those previously learned parameters to transform new data consistently.
 - so we apply fit transform on X_train and transform on X_test
- **Note: Of course you must do transforming but either 7 or 9**

```
[93]: X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

This is to show you how to do KNN choosing a specific number of neighbors Of course we must iterate over k like below and choose the best

```
[94]: knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, Y_train)
```

```
[94]: KNeighborsClassifier()
```

Accuracy: Number of total correct predictions the model got over all predictions.

```
[95]: y_pred = knn.predict(X_test)

accuracy = accuracy_score(Y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.884393063583815

Confusion Matrix: Table showing correct vs. incorrect predictions for each class.

```
[96]: confusion_matrix(Y_test, y_pred)
```

```
[96]: array([[ 62,   5,  16,   1],
          [  3,   5,   1,   2],
          [  2,   1, 231,   0],
          [  5,   4,   0,   8]], dtype=int64)
```

Classification Report: Summary of precision, recall, F1-score, and support for each class.

- Precision: Of all predicted positives, how many were actually correct.
- Recall: Of all actual positives, how many the model correctly identified.
- F1-score: Harmonic mean of precision and recall, balancing both.
- Support: Number of actual samples for each class in the dataset.

```
[97]: print(classification_report(Y_test, y_pred))
```

	precision	recall	f1-score	support
acc	0.86	0.74	0.79	84
good	0.33	0.45	0.38	11
unacc	0.93	0.99	0.96	234
vgood	0.73	0.47	0.57	17
accuracy			0.88	346
macro avg	0.71	0.66	0.68	346
weighted avg	0.89	0.88	0.88	346

Tenth Step: Best way to choose the number of nearest neighbors

1. loop over k from 1 to 21

2. apply KNeighborsClassifier on k
3. fit the model
4. predict using X_test
5. check accuracy
6. add accuracy to the array

```
[98]: k_values = range(1, 22)
      accuracies = []

      for k in k_values:
          knn = KNeighborsClassifier(n_neighbors=k)
          knn.fit(X_train, Y_train)
          y_pred = knn.predict(X_test)
          acc = accuracy_score(Y_test, y_pred)
          accuracies.append(acc)
```

Eleventh Step: Choose best k

1. find best k using maximum accuracy
2. get the best accuracy

```
[99]: best_k = k_values[np.argmax(accuracies)]
      best_accuracy = max(accuracies)
```

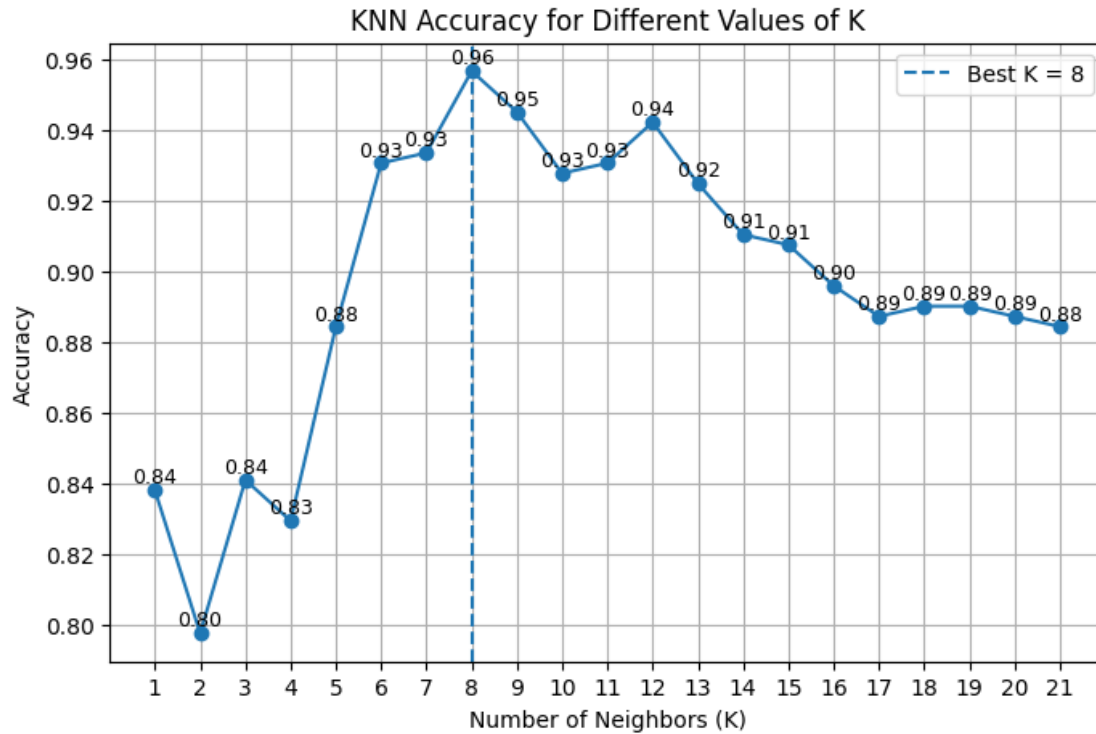
Twelfth Step: Plot the figure of k

- plot the accuracy of each k and highlight the best

```
[100]: plt.figure(figsize=(8, 5))
        plt.plot(k_values, accuracies, marker='o')
        plt.axvline(best_k, linestyle='--', label=f'Best K = {best_k}')

        # show accuracy values on the plot
        for k, acc in zip(k_values, accuracies):
            plt.text(k, acc + 0.002, f"{acc:.2f}", ha='center', fontsize=9)

        plt.xlabel("Number of Neighbors (K)")
        plt.ylabel("Accuracy")
        plt.title("KNN Accuracy for Different Values of K")
        plt.xticks(k_values)
        plt.legend()
        plt.grid(True)
        plt.show()
```



```
[101]: print("Best K:", best_k)
       print("Best Accuracy:", best_accuracy)
```

Best K: 8
Best Accuracy: 0.9566473988439307

Thirteenth Step: train the model using the best k

```
[102]: knn_best = KNeighborsClassifier(n_neighbors=best_k)
       knn_best.fit(X_train, Y_train)
```

```
[102]: KNeighborsClassifier(n_neighbors=8)
```

Fourteenth Step: predict using X_test and print metrics

```
[103]: y_pred_best = knn_best.predict(X_test)

       print("Final Model Accuracy:", accuracy_score(Y_test, y_pred_best))
       print("\nConfusion Matrix:\n", confusion_matrix(Y_test, y_pred_best))
       print("\nClassification Report:\n", classification_report(Y_test, y_pred_best))
```

Final Model Accuracy: 0.9566473988439307

Confusion Matrix:

```
[[ 76   2   5   1]
 [  1   9   0   1]
 [  2   0 232   0]
 [  3   0   0 14]]
```

Classification Report:

	precision	recall	f1-score	support
acc	0.93	0.90	0.92	84
good	0.82	0.82	0.82	11
unacc	0.98	0.99	0.99	234
vgood	0.88	0.82	0.85	17
accuracy			0.96	346
macro avg	0.90	0.88	0.89	346
weighted avg	0.96	0.96	0.96	346

Fifteenth Step: Save actual and Predicted values into a csv file

```
[104]: pred_df = pd.DataFrame({"Actual":Y_test, "Predicted":y_pred_best})
pred_df.to_csv("car_actual_vs_predicted_knn.csv", index=False)
pred_df
```

```
[104]:      Actual Predicted
599   unacc   unacc
1201    acc   unacc
628   unacc   unacc
1498    acc    acc
1263  unacc   unacc
...     ...      ...
100   unacc   unacc
274   unacc   unacc
1206  unacc   unacc
101   unacc   unacc
1084  unacc   unacc
```

```
[346 rows x 2 columns]
```

Sixteenth Step: Try a new unseen data

1. declare the data using array
2. Initialize X_df without target because it contains the columns names before scaling
3. Initialize new_car_df with the new data and columns names
4. transform the new_car_df
5. predict the target
6. print result either acc, unacc, good, vgood

```
[105]: # 1. Provide the new car data (matching the categorical values in the dataset)
# Order: [Price, Maintenance Cost, Doors, Persons, Luggage Boot, Security]
new_car = [50000, "vhigh", "4", "more", "big", "high"]

# 2. Setup X_df to get the correct column names for the DataFrame
X_df = df.drop("Acceptability", axis=1)

# 3. Create a DataFrame for the new car
new_car_df = pd.DataFrame(
    [new_car],
    columns=X_df.columns
)

# 4. Transform the data using the SAME preprocessor used during training
new_car_scaled = preprocessor.transform(new_car_df)

# 5. Use the best KNN model to predict
prediction = knn_best.predict(new_car_scaled)

# 6. Print the result (Acceptability: unacc, acc, good, or vgood)
print(f"Prediction: {prediction[0]}")
```

Prediction: unacc

```
[106]: import joblib
joblib.dump(knn_best, "knn_model.pkl")
joblib.dump(preprocessor, "preprocessor.pkl")
```

```
[106]: ['preprocessor.pkl']
```